

## Principal Components Analysis (PCA) y Sistemas de Recomendación.

Tutorial para la realización de métodos del Aprendizaje No Supervisado:

- Principal Components Analysis (PCA).
- Sistemas de Recomendación.

Hecho por: Estefania Nájera de la Rosa - [estefania.najera@udem.edu](mailto:estefania.najera@udem.edu) a 13 de abril del 2026.

## Principal Components Analysis (PCA).

Este método sirve para:

- Proyectar las características en componentes para tomar las principales según en % a cumplir.
- Reduce la dimensionalidad sin perder demasiada información.

Y es el que comenzaremos por desarrollar en el siguiente tutorial.

Tenemos que importar las librerías necesarias, estas siendo:

- **pandas** para el procesamiento de los datos.
- **numpy** para cálculos matemáticos.
- **matplotlib.pyplot** y **seaborn** para gráficas

En esta ocasión, estaremos trabajando con un dataset de cáncer de mama otorgado por la siguiente librería:

- **sklearn.datasets** con la declaración del dataset que es **load\_breast\_cancer**.

```
from sklearn.datasets import load_breast_cancer
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

Cargamos el dataset una vez que fue declarado.

```
# Cargar el dataset
breast_cancer = load_breast_cancer()
X = pd.DataFrame(breast_cancer.data, columns=breast_cancer.feature_names
y = pd.Series(breast_cancer.target, name='target')
```

### Características del Dataset 'Breast Cancer'.

Vamos a explorar las características principales del dataset, como el tamaño, el tipo de datos, las clases y la presencia de valores nulos.

```
print("### Dimensiones del dataset ###")
print(f"Dimensiones de las características (X): {X.shape}")
print(f"Dimensiones de la variable objetivo (y): {y.shape}\n")

print("### Tipos de datos de las características ###")
print(X.info())
print(f"\nTipos de datos de la variable objetivo (y): {y.dtype}\n")

print("### Clases en la variable objetivo ###")
print(f"Nombres de las clases: {breast_cancer.target_names}")
print("Distribución de las clases:")
print(y.value_counts())
print("\n")

print("### Verificación de valores nulos ###")
missing_values_X = X.isnull().sum()
missing_values_y = y.isnull().sum()

if missing_values_X.sum() == 0 and missing_values_y == 0:
    print("No hay valores nulos en el dataset.")
else:
    print("Valores nulos en X:\n", missing_values_X[missing_values_X > 0])
    print("Valores nulos en y:\n", missing_values_y)
```

```
### Dimensiones del dataset ###
Dimensiones de las características (X): (569, 30)
Dimensiones de la variable objetivo (y): (569,)
```

```
### Tipos de datos de las características ###
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 569 entries, 0 to 568
Data columns (total 30 columns):
#   Column                Non-Null Count  Dtype
---  -
0   mean radius            569 non-null    float64
1   mean texture           569 non-null    float64
```

```

2   mean perimeter      569 non-null    float64
3   mean area           569 non-null    float64
4   mean smoothness     569 non-null    float64
5   mean compactness    569 non-null    float64
6   mean concavity      569 non-null    float64
7   mean concave points  569 non-null    float64
8   mean symmetry       569 non-null    float64
9   mean fractal dimension 569 non-null    float64
10  radius error        569 non-null    float64
11  texture error       569 non-null    float64
12  perimeter error     569 non-null    float64
13  area error          569 non-null    float64
14  smoothness error    569 non-null    float64
15  compactness error   569 non-null    float64
16  concavity error     569 non-null    float64
17  concave points error 569 non-null    float64
18  symmetry error      569 non-null    float64
19  fractal dimension error 569 non-null    float64
20  worst radius        569 non-null    float64
21  worst texture       569 non-null    float64
22  worst perimeter     569 non-null    float64
23  worst area          569 non-null    float64
24  worst smoothness    569 non-null    float64
25  worst compactness   569 non-null    float64
26  worst concavity     569 non-null    float64
27  worst concave points 569 non-null    float64
28  worst symmetry      569 non-null    float64
29  worst fractal dimension 569 non-null    float64
dtypes: float64(30)
memory usage: 133.5 KB
None

```

Tipos de datos de la variable objetivo (y): int64

```

### Clases en la variable objetivo ###
Nombres de las clases: ['malignant' 'benign']
Distribución de las clases:
target
1      357
0      212
Name: count, dtype: int64

```

```

### Verificación de valores nulos ###
No hay valores nulos en el dataset.

```

Aquí podemos observar que todos nuestros datos son de tipo flotante, por lo que no hay que hacer ajustes de variables categóricas a numéricas.

### Análisis Univariable: Gráficas de Densidad.

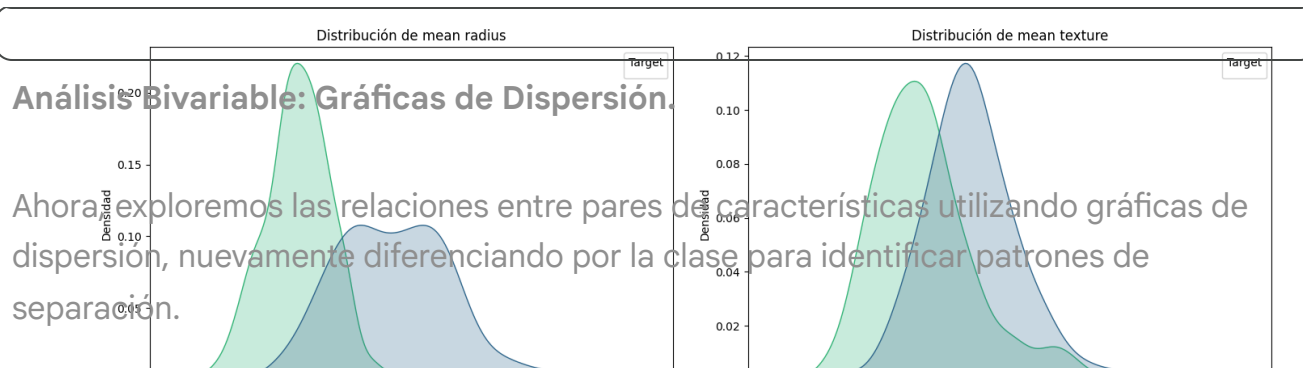
Vamos a visualizar la distribución de algunas de las características más importantes utilizando gráficas de densidad, separando por la clase (maligno vs. benigno) para observar posibles diferencias.

```
selected_features_uni = ['mean radius', 'mean texture', 'mean perimeter']

plt.figure(figsize=(15, 10))
for i, feature in enumerate(selected_features_uni):
    plt.subplot(2, 2, i + 1)
    # Let seaborn handle the legend generation directly
    sns.kdeplot(data=X, x=feature, hue=y, fill=True, common_norm=False,
                plt.title(f'Distribución de {feature}'))
    plt.xlabel(feature)
    plt.ylabel('Densidad')
    # Customize legend labels after seaborn has created them
    current_handles, current_labels = plt.gca().get_legend_handles_label
    new_labels = [breast_cancer.target_names[int(float(label))]] for labe
    plt.gca().legend(current_handles, new_labels, title='Target')

plt.tight_layout()
plt.show()
```





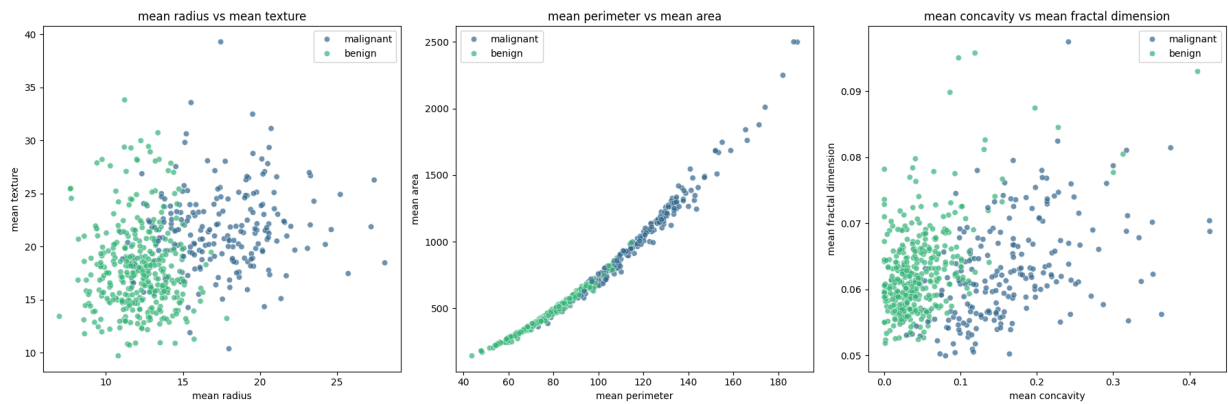
## Análisis Bivariable: Gráficas de Dispersión.

Ahora exploremos las relaciones entre pares de características utilizando gráficas de dispersión, nuevamente diferenciando por la clase para identificar patrones de separación.

```
selected_features_bi = [('mean radius', 'mean texture'), ('mean perimete

plt.figure(figsize=(18, 6))
for i, (feature1, feature2) in enumerate(selected_features_bi):
    plt.subplot(1, 3, i + 1)
    sns.scatterplot(x=X[feature1], y=X[feature2], hue=y, palette='viridi
    plt.title(f'{feature1} vs {feature2}')
    plt.xlabel(feature1)
    plt.ylabel(feature2)
    handles, labels = plt.gca().get_legend_handles_labels()
    plt.gca().legend(handles=handles, labels=[breast_cancer.target_names

plt.tight_layout()
plt.show()
```



## Preparación de Datos: División y Estandarización.

Dividiremos el dataset en conjuntos de entrenamiento y prueba para evaluar el rendimiento del modelo. Luego, estandarizaremos las características para asegurar que todas contribuyan de manera equitativa al modelo.

Importamos **sklearn.model\_selection** con **train\_test\_split** para dividir, seguido de **sklearn.preprocessing** con **StandardScaler** para estandarizar los datos.

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# Dividir el dataset en conjuntos de entrenamiento y prueba
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,

print(f"Dimensiones de X_train: {X_train.shape}")
```

```
print(f"Dimensiones de X_test: {X_test.shape}")
print(f"Dimensiones de y_train: {y_train.shape}")
print(f"Dimensiones de y_test: {y_test.shape}")
```

```
Dimensiones de X_train: (398, 30)
Dimensiones de X_test: (171, 30)
Dimensiones de y_train: (398,)
Dimensiones de y_test: (171,)
```

```
# Estandarizar las características
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print("Datos estandarizados correctamente.")
# Mostrar las primeras 5 filas de los datos de entrenamiento estandariza
print("\nPrimeras 5 filas de X_train_scaled (estandarizado):\n")
print(pd.DataFrame(X_train_scaled, columns=X.columns).head())
```

Datos estandarizados correctamente.

Primeras 5 filas de X\_train\_scaled (estandarizado):

|   | mean radius | mean texture | mean perimeter | mean area | mean smoothness |
|---|-------------|--------------|----------------|-----------|-----------------|
| 0 | -0.709821   | -0.258417    | -0.637396      | -0.711996 | 1.628430        |
| 1 | -0.830331   | 2.231127     | -0.874980      | -0.779081 | -1.597569       |
| 2 | -1.011097   | -0.227270    | -1.035172      | -0.911780 | -1.077779       |
| 3 | -0.382721   | -0.111581    | -0.408966      | -0.453660 | 0.072810        |
| 4 | -0.804508   | -1.401961    | -0.810071      | -0.773196 | 0.191319        |

|   | mean compactness | mean concavity | mean concave points | mean symmetry |
|---|------------------|----------------|---------------------|---------------|
| 0 | 0.847289         | 0.166501       | 0.196420            | 0.542716      |
| 1 | -1.267717        | -1.086471      | -1.243232           | -2.658543     |
| 2 | -1.098096        | -0.841650      | -0.955767           | 2.023790      |
| 3 | -0.443255        | -0.756276      | -0.782409           | -0.163916     |
| 4 | -0.507801        | -0.685397      | -0.530392           | -0.128228     |

|   | mean fractal dimension | ... | worst radius | worst texture | worst perime |
|---|------------------------|-----|--------------|---------------|--------------|
| 0 | 1.347151               | ... | -0.600068    | -0.065867     | -0.565       |
| 1 | -1.052699              | ... | -0.897361    | 1.975153      | -0.949       |
| 2 | -0.688547              | ... | -1.099685    | -0.396544     | -1.122       |
| 3 | -0.120413              | ... | -0.519552    | 0.027482      | -0.575       |
| 4 | -0.001311              | ... | -0.814779    | -1.524644     | -0.858       |

|   | worst area | worst smoothness | worst compactness | worst concavity | \ |
|---|------------|------------------|-------------------|-----------------|---|
| 0 | -0.622613  | 2.019222         | 0.186202          | 0.180187        |   |
| 1 | -0.779331  | -1.747652        | -1.229372         | -1.258947       |   |
| 2 | -0.909132  | -1.515009        | -1.165663         | -1.078897       |   |
| 3 | -0.531239  | -0.108556        | -0.445414         | -0.692056       |   |
| 4 | -0.747633  | 0.156312         | -0.647239         | -0.683022       |   |

|   | worst concave points | worst symmetry | worst fractal dimension |
|---|----------------------|----------------|-------------------------|
| 0 | 0.400933             | -0.414074      | 0.445223                |
| 1 | -1.692788            | -2.095723      | -1.323417               |



|   |           |           |           |
|---|-----------|-----------|-----------|
| 2 | -1.364240 | -0.354125 | -0.886645 |
| 3 | -0.463762 | -0.284953 | -0.622794 |
| 4 | -0.398999 | -0.301862 | -0.210724 |

[5 rows x 30 columns]

## Modelado: Regresión Logística y Evaluación.

Ahora vamos a entrenar un modelo de Regresión Logística con los datos estandarizados, y luego evaluaremos su rendimiento utilizando métricas clave como Accuracy, F1-Score y AUC.

Es importante mencionar que para la elaboración de este modelo de clasificación, necesitamos importar su librería:

- **sklearn.linear\_model** con la declaración del algoritmo que es **LogisticRegression**.

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, f1_score, roc_auc_score, con

# Inicializar y entrenar el modelo de Regresión Logística
# Se usa random_state para reproducibilidad y solver='liblinear' es buen
model = LogisticRegression(random_state=42, solver='liblinear') # Regula
model.fit(X_train_scaled, y_train)

print("Modelo de Regresión Logística entrenado correctamente.")
```

Modelo de Regresión Logística entrenado correctamente.

```
# Realizar predicciones sobre el conjunto de prueba estandarizado
y_pred = model.predict(X_test_scaled)
y_pred_proba = model.predict_proba(X_test_scaled)[: , 1] # Probabilidades

print("Predicciones realizadas.")
```

Predicciones realizadas.

```
# Evaluar el rendimiento del modelo
accuracy = accuracy_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)
roc_auc = roc_auc_score(y_test, y_pred_proba)

print(f"### Métricas del Modelo de Regresión Logística ###")
print(f"Accuracy: {accuracy:.4f}")
print(f"F1-Score: {f1:.4f}")
print(f"AUC: {roc_auc:.4f}")
```

```
print("\n### Matriz de Confusión ###")
print(confusion_matrix(y_test, y_pred))

print("\n### Reporte de Clasificación ###")
print(classification_report(y_test, y_pred))
```

```
### Métricas del Modelo de Regresión Logística ###
```

```
Accuracy: 0.9883
```

```
F1-Score: 0.9907
```

```
AUC: 0.9981
```

```
### Matriz de Confusión ###
```

```
[[ 63   1]
```

```
 [  1 106]]
```

```
### Reporte de Clasificación ###
```

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 0.98      | 0.98   | 0.98     | 64      |
| 1            | 0.99      | 0.99   | 0.99     | 107     |
| accuracy     |           |        | 0.99     | 171     |
| macro avg    | 0.99      | 0.99   | 0.99     | 171     |
| weighted avg | 0.99      | 0.99   | 0.99     | 171     |

Los resultados del modelo nos indican lo siguiente:

### Matriz de Confusión:

- 169 aciertos.
- 2 errores.

**171 datos en total.**

### Reporte de Clasificación:

- Con 64 datos de la clase 0, se obtiene:
  - 98% de positivos encontrados (recall).
  - 98% fueron clasificados correctamente (precision).
  - 98% es el promedio ponderado de las clases (f1-score).
- Con 107 datos de la clase 1, se obtiene:
  - 99% de positivos encontrados.
  - 99% fueron clasificados correctamente.
  - 99% es el promedio ponderado de las clases (f1-score).

El modelo tiene un accuracy de 99%.

## Visualización de la Curva ROC (Receiver Operating Characteristic).

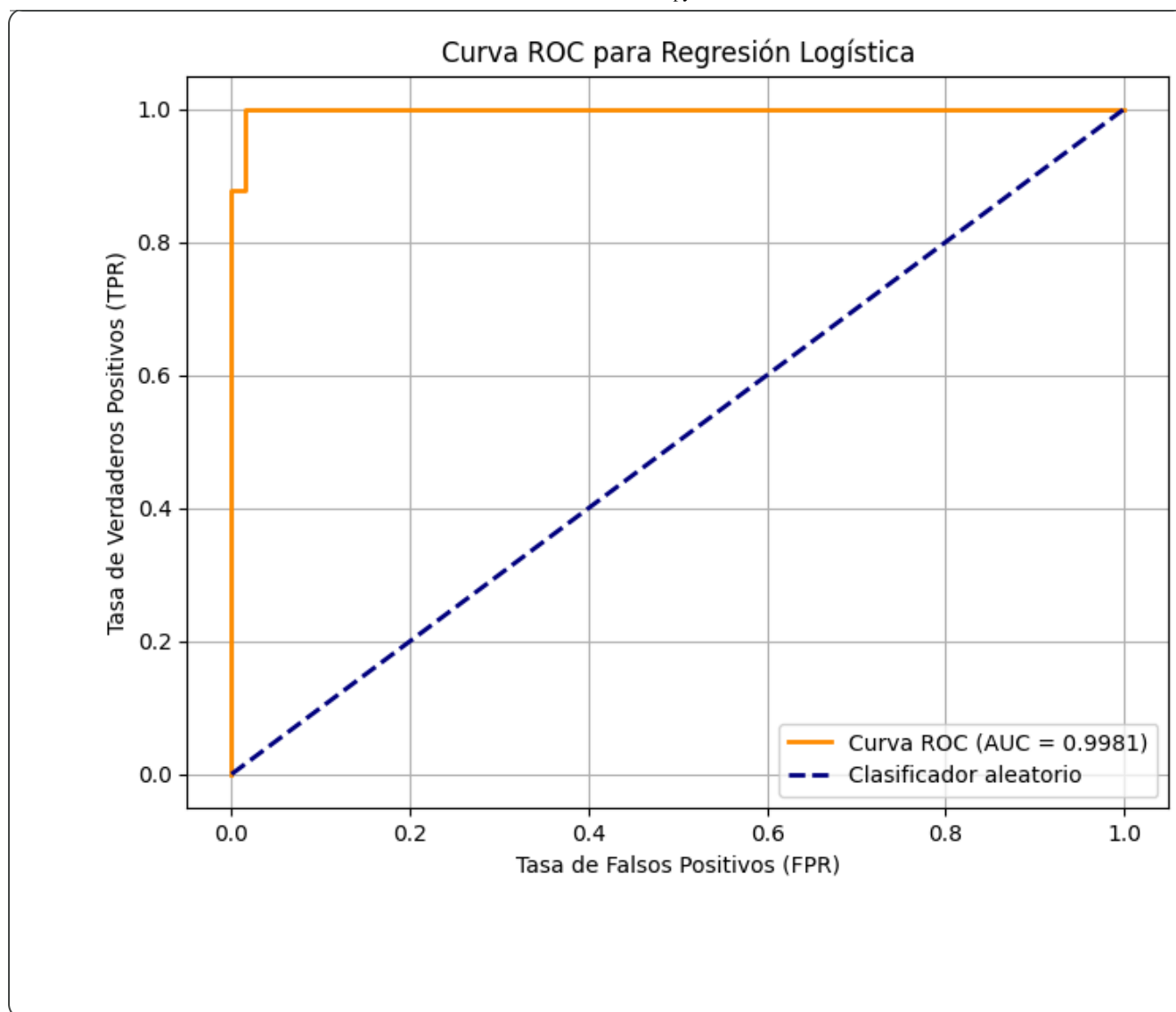
Vamos a graficar la curva ROC para el modelo de Regresión Logística original para tener una representación visual de su rendimiento.

Importamos **sklearn.metrics** con sus declaraciones de **roc\_curve** y **auc**.

```
from sklearn.metrics import roc_curve, auc

# Calcular FPR, TPR y umbrales
fpr, tpr, thresholds = roc_curve(y_test, y_pred_proba)
roc_auc = auc(fpr, tpr)

# Graficar la curva ROC
plt.figure(figsize=(8, 6))
plt.plot(fpr, tpr, color='darkorange', lw=2, label=f'Curva ROC (AUC = {r
plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--', label='Clas
plt.xlabel('Tasa de Falsos Positivos (FPR)')
plt.ylabel('Tasa de Verdaderos Positivos (TPR)')
plt.title('Curva ROC para Regresión Logística')
plt.legend(loc='lower right')
plt.grid(True)
plt.show()
```



## Análisis de Componentes Principales (PCA).

Ahora aplicaremos PCA a nuestros datos estandarizados para reducir la dimensionalidad y entender las principales fuentes de variabilidad. Analizaremos la varianza explicada por cada componente principal y visualizaremos los resultados.

Importamos **sklearn.decomposition** con la declaración del método de **PCA**.

```
from sklearn.decomposition import PCA

# Inicializar PCA y ajustarlo a los datos de entrenamiento escalados
pca = PCA(n_components=None) # n_components=None para mantener todas las
pca.fit(X_train_scaled)

# Varianza explicada por cada componente
explained_variance_ratio = pca.explained_variance_ratio_

# Suma acumulada de la varianza explicada
```

```
cum_explained_variance = np.cumsum(explained_variance_ratio)

print("Varianza explicada por cada componente principal:\n", explained_v
print("\nVarianza acumulada explicada:\n", cum_explained_variance[:5]) #
```

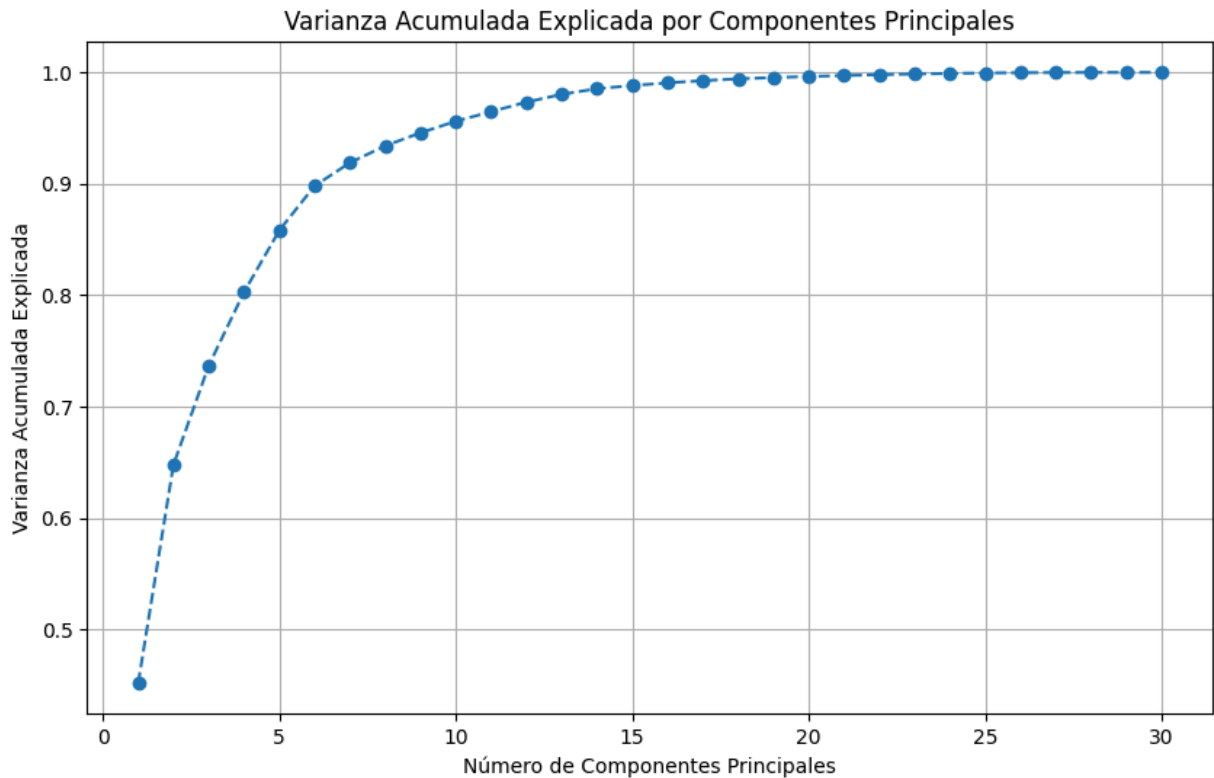
```
Varianza explicada por cada componente principal:
[0.45156229 0.19628669 0.08897898 0.06596846 0.05540815]
```

```
Varianza acumulada explicada:
[0.45156229 0.64784898 0.73682795 0.80279641 0.85820456]
```

## Visualización de la Varianza Explicada.

Grafiquemos la varianza explicada acumulada para determinar cuántos componentes principales retener.

```
plt.figure(figsize=(10, 6))
plt.plot(range(1, len(cum_explained_variance) + 1), cum_explained_varian
plt.title('Varianza Acumulada Explicada por Componentes Principales')
plt.xlabel('Número de Componentes Principales')
plt.ylabel('Varianza Acumulada Explicada')
plt.grid(True)
plt.show()
```



Vale la pena mencionar que aquí es donde podemos delimitar cuántos componentes necesitamos para explicar cierto porcentaje que se establezca.

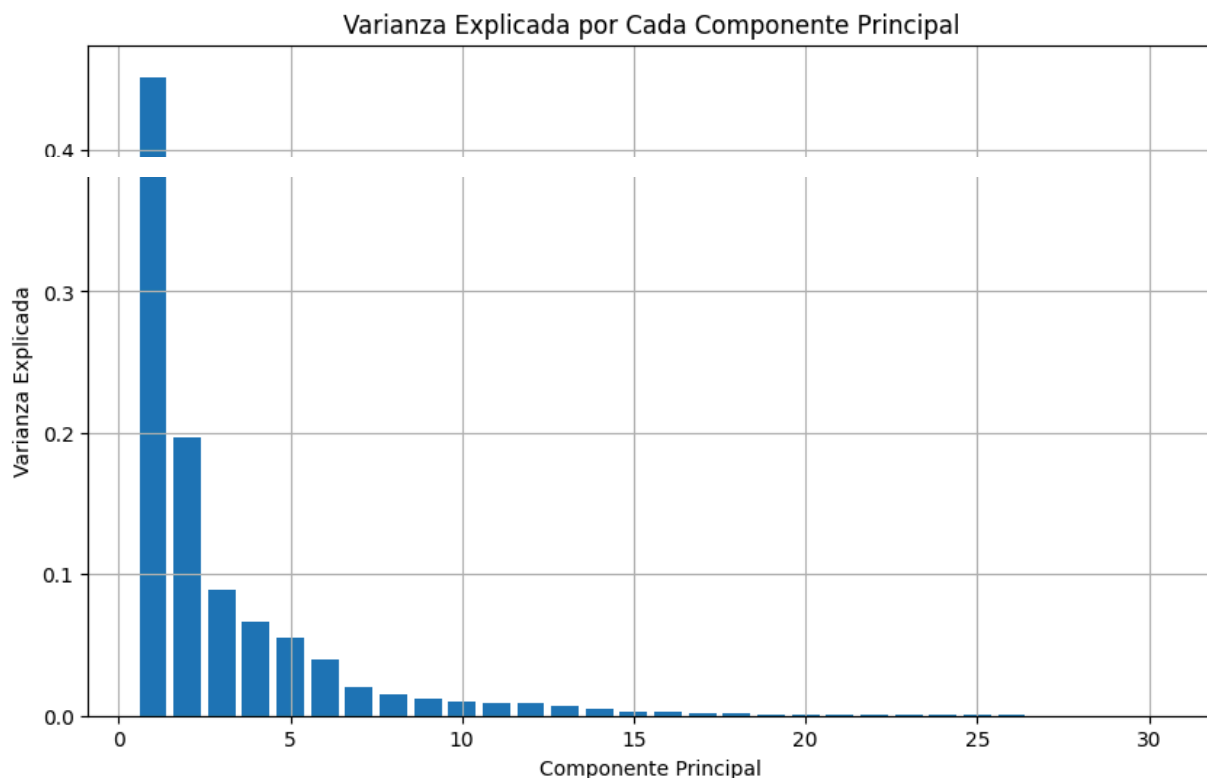
Por ejemplo, si queremos explicar hasta un 80%, podemos hacerlo con alrededor de 4 componentes, mientras que el explicar un 60%-70% se puede hacer con 2 o 3 componentes. Esto se va a profundizar más adelante con exactitud.

### Varianza Explicada por Componente Principal.

Vamos a visualizar la varianza explicada individualmente por cada componente principal para identificar un "codo" que sugiera un número óptimo de componentes.

```
plt.figure(figsize=(10, 6))
plt.bar(range(1, len(explained_variance_ratio) + 1), explained_variance_ratio)
plt.title('Varianza Explicada por Cada Componente Principal')
plt.xlabel('Componente Principal')
plt.ylabel('Varianza Explicada')
```

```
plt.grid(True)
plt.show()
```



### Identificación de Componentes para Varianza Acumulada.

Basándonos en la gráfica de varianza acumulada, podemos determinar el número de componentes necesarios para explicar un porcentaje deseado de la varianza total de los datos. Esto nos ayudará a elegir el '**n\_components**' para el entrenamiento de nuestros modelos.

Aquí identificamos los componentes para el 60%, 70% y 80% de la varianza explicada:

```
def find_n_components(cumulative_variance, target_variance):
    for i, var in enumerate(cumulative_variance):
        if var >= target_variance:
            return i + 1
    return len(cumulative_variance)
```

```

n_components_60 = find_n_components(cum_explained_variance, 0.60)
n_components_70 = find_n_components(cum_explained_variance, 0.70)
n_components_80 = find_n_components(cum_explained_variance, 0.80)
n_components_90 = find_n_components(cum_explained_variance, 0.90)

print(f"Para explicar el 60% de la varianza, se necesitan {n_components_60} componentes.")
print(f"Para explicar el 70% de la varianza, se necesitan {n_components_70} componentes.")
print(f"Para explicar el 80% de la varianza, se necesitan {n_components_80} componentes.")
print(f"Para explicar el 90% de la varianza, se necesitan {n_components_90} componentes.")

# Definir una lista de n_components a probar, incluyendo el modelo original
components_to_test = [n_components_60, n_components_70, n_components_80, n_components_90]
components_to_test = sorted(list(set(components_to_test))) # Eliminar duplicados

Para explicar el 60% de la varianza, se necesitan 2 componentes.
Para explicar el 70% de la varianza, se necesitan 3 componentes.
Para explicar el 80% de la varianza, se necesitan 4 componentes.
Para explicar el 90% de la varianza, se necesitan 7 componentes.

```

## Modelado y Comparación con Diferentes Componentes PCA.

Ahora, vamos a entrenar y evaluar modelos de Regresión Logística utilizando el número de componentes PCA identificados para diferentes niveles de varianza explicada. Compararemos los resultados con el modelo original (sin PCA) para entender el impacto de la reducción de dimensionalidad.

```

results = {}

# Función para entrenar y evaluar el modelo con un número dado de componentes
def train_and_evaluate_pca_model(n_comp):
    if n_comp == 0: # Caso para el modelo sin PCA (usando todos los componentes)
        # Se usará el modelo previamente entrenado sin PCA
        y_pred = model.predict(X_test_scaled)
        y_pred_proba = model.predict_proba(X_test_scaled)[:, 1]
    else:
        pca_temp = PCA(n_components=n_comp, random_state=42) # random_state=42
        X_train_pca_temp = pca_temp.fit_transform(X_train_scaled)
        X_test_pca_temp = pca_temp.transform(X_test_scaled)

        model_pca = LogisticRegression(random_state=42, solver='liblinear')
        model_pca.fit(X_train_pca_temp, y_train)

        y_pred = model_pca.predict(X_test_pca_temp)
        y_pred_proba = model_pca.predict_proba(X_test_pca_temp)[:, 1]

    accuracy = accuracy_score(y_test, y_pred)
    f1 = f1_score(y_test, y_pred)
    roc_auc = roc_auc_score(y_test, y_pred_proba)

```



```

    return {'Accuracy': accuracy, 'F1-Score': f1, 'AUC': roc_auc}

# Evaluar el modelo original (sin PCA, ya entrenado)
results['Original (No PCA)'] = train_and_evaluate_pca_model(0) # Usar 0

# Evaluar modelos con diferentes números de componentes PCA
for n_comp in components_to_test:
    print(f"Entrenando modelo con {n_comp} componentes PCA...")
    results[f'{n_comp} Components (PCA)'] = train_and_evaluate_pca_model

# Convertir resultados a DataFrame para una mejor visualización
results_df = pd.DataFrame.from_dict(results, orient='index')

print("\n### Comparación de Rendimiento de Modelos ###")
display(results_df.sort_values(by='F1-Score', ascending=False))

```

```

Entrenando modelo con 2 componentes PCA...
Entrenando modelo con 3 componentes PCA...
Entrenando modelo con 4 componentes PCA...
Entrenando modelo con 7 componentes PCA...

```

```
### Comparación de Rendimiento de Modelos ###
```

|                           | Accuracy | F1-Score | AUC      |
|---------------------------|----------|----------|----------|
| <b>Original (No PCA)</b>  | 0.988304 | 0.990654 | 0.998102 |
| <b>4 Components (PCA)</b> | 0.959064 | 0.968037 | 0.995765 |
| <b>7 Components (PCA)</b> | 0.959064 | 0.966825 | 0.995911 |
| <b>2 Components (PCA)</b> | 0.953216 | 0.962617 | 0.992261 |
| <b>3 Components (PCA)</b> | 0.912281 | 0.929577 | 0.988172 |



## Heatmaps de Correlación: Antes y Después de PCA.

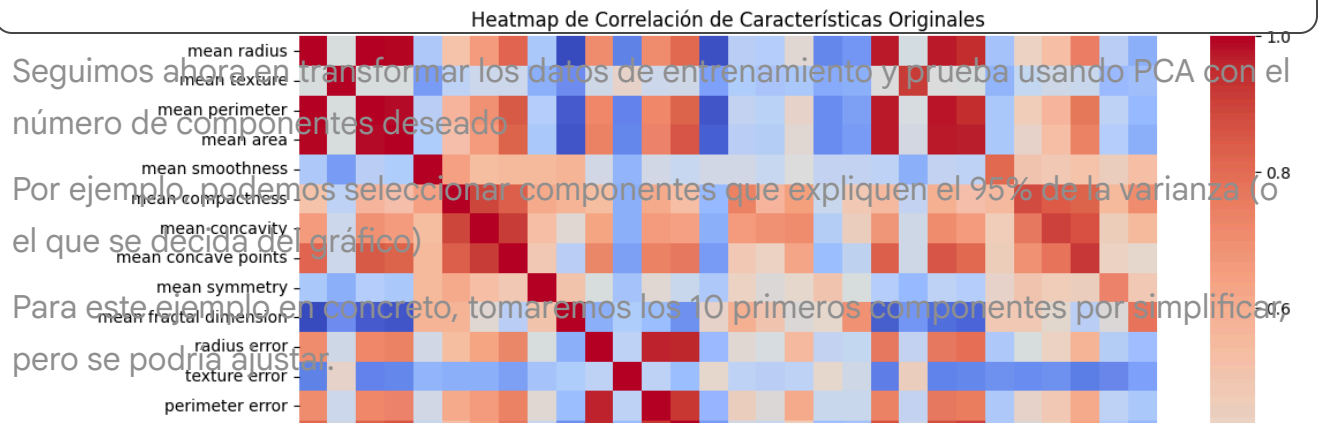
Vamos a visualizar la matriz de correlación de las características originales y luego cómo se comportan las correlaciones de las componentes principales. Esto nos ayudará a entender cómo PCA descorrelaciona las variables.

```

# Correlación de las características originales
plt.figure(figsize=(12, 10))
sns.heatmap(X_train.corr(), annot=False, cmap='coolwarm')
plt.title('Heatmap de Correlación de Características Originales')
plt.show()

```





Seguimos ahora en transformar los datos de entrenamiento y prueba usando PCA con el número de componentes deseado.

Por ejemplo, podemos seleccionar componentes que expliquen el 95% de la varianza (o el que se decida del gráfico).

Para este ejemplo en concreto, tomaremos los 10 primeros componentes por simplificar, pero se podría ajustar.

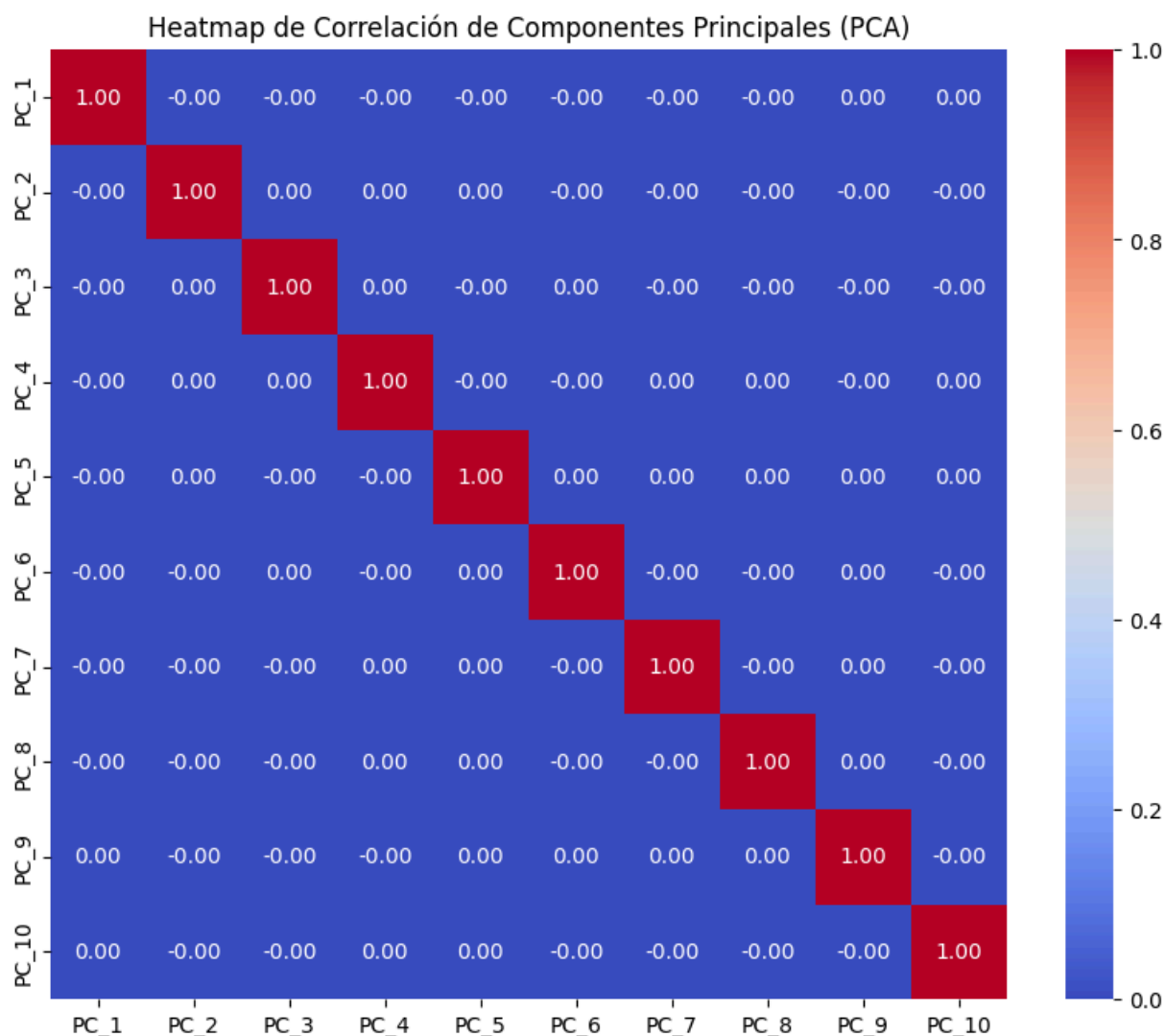
```
# Vamos a re-inicializar PCA para un número fijo de componentes basado en
n_components_to_retain = 10 # Se puede ajustar según el gráfico de varianza
pca_final = PCA(n_components=n_components_to_retain)

X_train_pca = pca_final.fit_transform(X_train_scaled)
X_test_pca = pca_final.transform(X_test_scaled)

# Crear un DataFrame para las componentes principales transformadas (para
pca_df = pd.DataFrame(X_train_pca, columns=[f'PC_{i+1}' for i in range(n

# Correlación de las Componentes Principales
plt.figure(figsize=(10, 8))
sns.heatmap(pca_df.corr(), annot=True, cmap='coolwarm', fmt=".2f")
plt.title('Heatmap de Correlación de Componentes Principales (PCA)')
plt.show()
```

PC\_1  
PC\_2  
PC\_3  
PC\_4  
PC\_5  
PC\_6  
PC\_7  
PC\_8  
PC\_9  
PC\_10  
PC\_11  
PC\_12  
PC\_13  
PC\_14  
PC\_15  
PC\_16  
PC\_17  
PC\_18  
PC\_19  
PC\_20  
PC\_21  
PC\_22  
PC\_23  
PC\_24  
PC\_25  
PC\_26  
PC\_27  
PC\_28  
PC\_29  
PC\_30  
PC\_31  
PC\_32  
PC\_33  
PC\_34  
PC\_35  
PC\_36  
PC\_37  
PC\_38  
PC\_39  
PC\_40  
PC\_41  
PC\_42  
PC\_43  
PC\_44  
PC\_45  
PC\_46  
PC\_47  
PC\_48  
PC\_49  
PC\_50  
PC\_51  
PC\_52  
PC\_53  
PC\_54  
PC\_55  
PC\_56  
PC\_57  
PC\_58  
PC\_59  
PC\_60  
PC\_61  
PC\_62  
PC\_63  
PC\_64  
PC\_65  
PC\_66  
PC\_67  
PC\_68  
PC\_69  
PC\_70  
PC\_71  
PC\_72  
PC\_73  
PC\_74  
PC\_75  
PC\_76  
PC\_77  
PC\_78  
PC\_79  
PC\_80  
PC\_81  
PC\_82  
PC\_83  
PC\_84  
PC\_85  
PC\_86  
PC\_87  
PC\_88  
PC\_89  
PC\_90  
PC\_91  
PC\_92  
PC\_93  
PC\_94  
PC\_95  
PC\_96  
PC\_97  
PC\_98  
PC\_99  
PC\_100

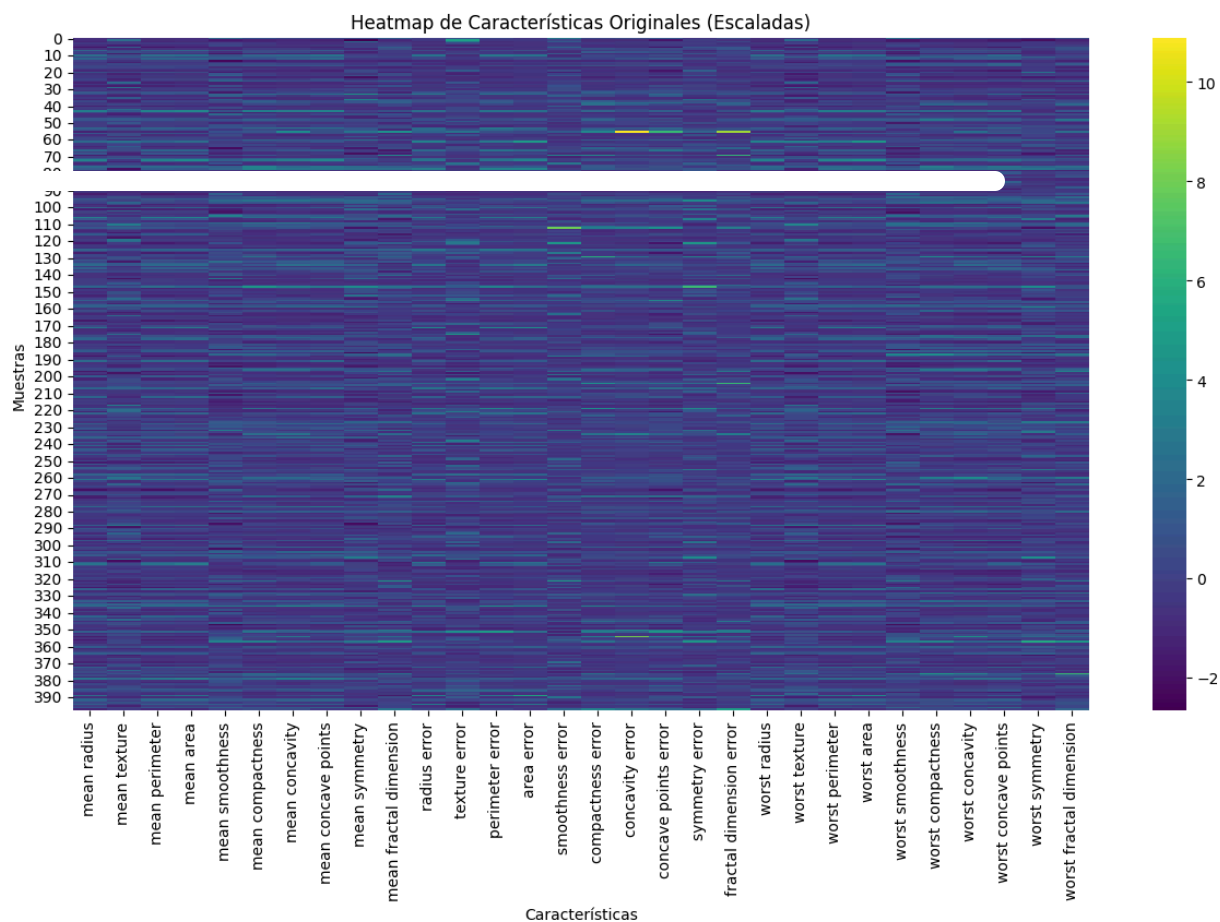


### Visualización de los Datos: Antes y Después de PCA.

Para entender mejor cómo PCA transforma los datos, vamos a visualizar las matrices de las características originales (escaladas) y de las componentes principales resultantes. Esto nos permitirá observar directamente la reorganización de la información.

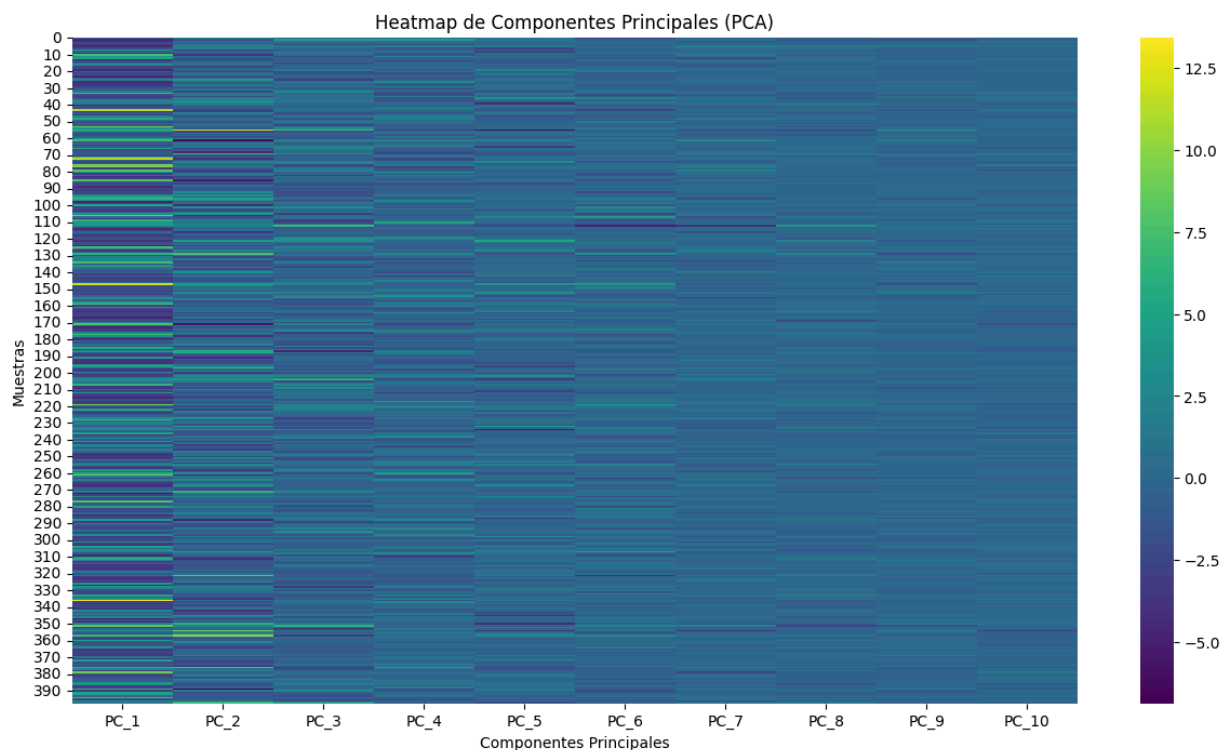
```
# Visualización de las características originales (escaladas)
plt.figure(figsize=(15, 8))
sns.heatmap(pd.DataFrame(X_train_scaled, columns=X.columns), cmap='virid
plt.title('Heatmap de Características Originales (Escaladas)')
plt.xlabel('Características')
```

```
plt.ylabel('Muestras')
plt.show()
```



```
# Visualización de las componentes principales transformadas
plt.figure(figsize=(15, 8))
```

```
sns.heatmap(pca_df, cmap='viridis', cbar=True)
plt.title('Heatmap de Componentes Principales (PCA)')
plt.xlabel('Componentes Principales')
plt.ylabel('Muestras')
plt.show()
```



## Reporte de Clasificación para Modelo con PCA.

Vamos a generar el reporte de clasificación para el modelo de Regresión Logística que utiliza 4 componentes principales, el cual mostró un buen rendimiento. Esto nos dará un desglose detallado de la precision, recall y F1-score para cada clase.

```
from sklearn.decomposition import PCA
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report

n_components_for_report = 4 # Elegimos 4 componentes por su buen rendimi

# Reaplicar PCA con el número de componentes elegido
pca_report = PCA(n_components=n_components_for_report, random_state=42)
X_train_pca_report = pca_report.fit_transform(X_train_scaled)
X_test_pca_report = pca_report.transform(X_test_scaled)

# Entrenar el modelo de Regresión Logística con los datos PCA-transforma
model_pca_report = LogisticRegression(random_state=42, solver='liblinear')
model_pca_report.fit(X_train_pca_report, y_train)

# Realizar predicciones
y_pred_pca_report = model_pca_report.predict(X_test_pca_report)

print(f"### Reporte de Clasificación para Modelo con {n_components_for_r")
print(classification_report(y_test, y_pred_pca_report, target_names=brea
```

```
### Reporte de Clasificación para Modelo con 4 Componentes PCA ###
              precision    recall  f1-score   support

    malignant      0.98      0.91      0.94         64
      benign      0.95      0.99      0.97        107

   accuracy              0.96         171
  macro avg      0.96      0.95      0.96         171
 weighted avg      0.96      0.96      0.96         171
```

Los resultados del modelo nos indican lo siguiente:

### Reporte de Clasificación:

- Con 64 datos de la clase maligna (malignant), se obtiene:
  - 91% de positivos encontrados (recall).
  - 98% fueron clasificados correctamente (precision).
  - 94% es el promedio ponderado de las clases (f1-score).
- Con 107 datos de la clase benigna (benign), se obtiene:
  - 99% de positivos encontrados.

- 95% fueron clasificados correctamente.
- 97% es el promedio ponderado de las clases (f1-score).

El modelo tiene un accuracy de 96%.

## Tabla Comparativa de Rendimiento con PCA por Varianza Explicada.

Vamos a consolidar los resultados en una tabla para comparar el rendimiento del modelo de Regresión Logística para diferentes niveles de varianza explicada por PCA.

```
# Encontrar el número de componentes para el 95% de la varianza si aún n
n_components_95 = find_n_components(cum_explained_variance, 0.95)

# Añadir a la lista de componentes a probar si no existe
if n_components_95 not in components_to_test:
    components_to_test.append(n_components_95)
    components_to_test = sorted(list(set(components_to_test))) # Asegura
    print(f"Se añadió {n_components_95} componentes para el 95% de la va

# Re-evaluar los modelos si es necesario para asegurar que todos los com
for n_comp in components_to_test:
    if f'{n_comp} Components (PCA)' not in results:
        print(f"Entrenando modelo con {n_comp} componentes PCA (para el
        results[f'{n_comp} Components (PCA)'] = train_and_evaluate_pca_m

# Preparar los datos para la tabla comparativa
data_for_table = []

# Agregar el modelo original (sin PCA)
data_for_table.append({
    'Tipo de Modelo': 'Original (No PCA)',
    '% Varianza Explicada': 'N/A',
    'Nº Componentes': X_train_scaled.shape[1],
    'Accuracy': results['Original (No PCA)']['Accuracy'],
    'F1-Score': results['Original (No PCA)']['F1-Score'],
    'AUC': results['Original (No PCA)']['AUC']
})

# Agregar los modelos con PCA
for n_comp in components_to_test:
    # Buscar la varianza explicada acumulada para este número de compone
    # Asegurarse de que el índice no exceda el tamaño del array
    explained_var = cum_explained_variance[n_comp - 1] if n_comp <= len(

    data_for_table.append({
        'Tipo de Modelo': f'{n_comp} Componentes (PCA)',
        '% Varianza Explicada': f'{explained_var:.2%}',
        'Nº Componentes': n_comp,
```



```

        'Accuracy': results[f'{n_comp} Components (PCA)']['Accuracy'],
        'F1-Score': results[f'{n_comp} Components (PCA)']['F1-Score'],
        'AUC': results[f'{n_comp} Components (PCA)']['AUC']
    })

summary_df = pd.DataFrame(data_for_table)

print("### Resumen de Rendimiento de Modelos con y sin PCA ###")
display(summary_df.sort_values(by='F1-Score', ascending=False))

```

Se añadió 10 componentes para el 95% de la varianza explicada a la lista  
Entrenando modelo con 10 componentes PCA (para el 95% de la varianza)...

### Resumen de Rendimiento de Modelos con y sin PCA ###

|   | Tipo de Modelo       | % Varianza Explicada | Nº Componentes | Accuracy | F1-Score | AUC      |  |
|---|----------------------|----------------------|----------------|----------|----------|----------|--|
| 0 | Original (No PCA)    | N/A                  | 30             | 0.988304 | 0.990654 | 0.998102 |  |
| 5 | 10 Componentes (PCA) | 95.62%               | 10             | 0.970760 | 0.976303 | 0.997956 |  |
| 3 | 4 Componentes (PCA)  | 80.28%               | 4              | 0.959064 | 0.968037 | 0.995765 |  |

## Visualización de Componentes Principales para Clasificación.

Vamos a generar gráficas de densidad y dispersión utilizando los componentes principales para observar cómo se separan las clases (maligno vs. benigno) en este espacio de dimensionalidad reducida.

## Gráficas de Densidad Univariable con Componentes Principales.

Visualizaremos la distribución de las primeras componentes principales, separando por la clase objetivo.

Es importante mencionar que para las gráficas de densidad, usaremos los datos de entrenamiento transformados y su etiqueta y así combinar `X_train_pca` y `y_train` en un `DataFrame` para facilitar la graficación.

```

pca_train_df = pd.DataFrame(X_train_pca, columns=[f'PC_{i+1}' for i in range(X_train_pca.shape[1]-1)])
pca_train_df['target'] = y_train.values # Asegurarse de que el índice coincida

selected_pcs_uni = ['PC_1', 'PC_2', 'PC_3'] # Seleccionar las primeras 3 componentes

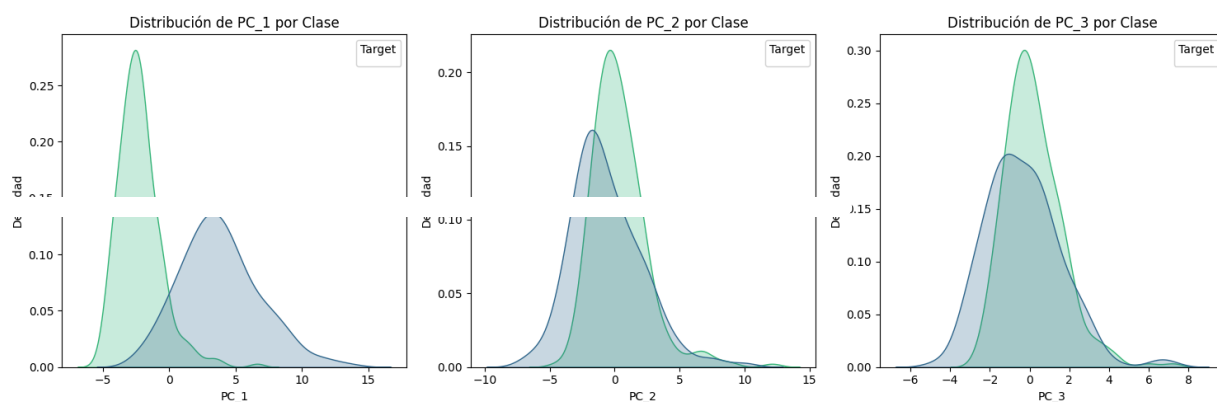
```

```

plt.figure(figsize=(15, 5))
for i, pc in enumerate(selected_pcs_uni):
    plt.subplot(1, 3, i + 1)
    sns.kdeplot(data=pca_train_df, x=pc, hue='target', fill=True, common_
plt.title(f'Distribución de {pc} por Clase')
plt.xlabel(pc)
plt.ylabel('Densidad')
# Personalizar las etiquetas de la leyenda
current_handles, current_labels = plt.gca().get_legend_handles_label
new_labels = [breast_cancer.target_names[int(float(label))]] for labe
plt.gca().legend(current_handles, new_labels, title='Target')

plt.tight_layout()
plt.show()

```



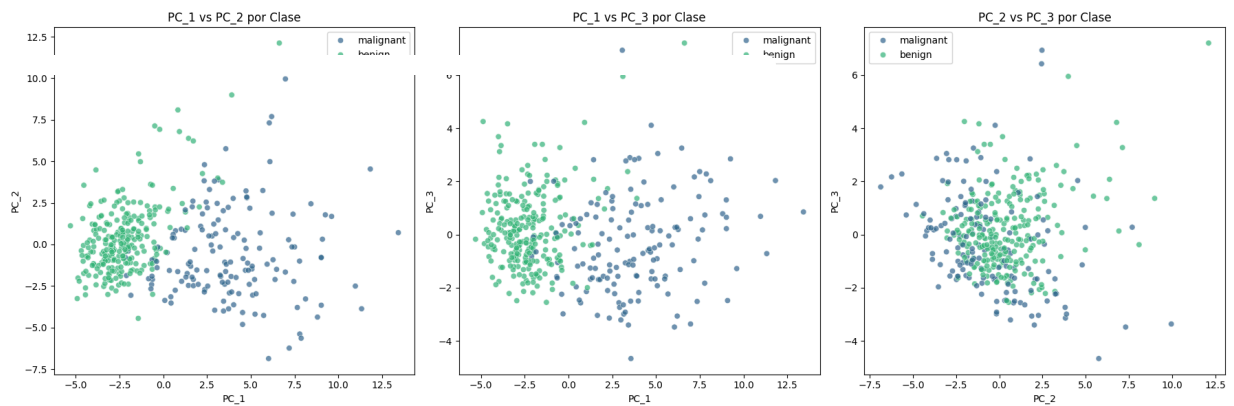
## Gráficas de Dispersión Bivariable con Componentes Principales.

Ahora, exploremos las relaciones entre pares de componentes principales, diferenciando por la clase objetivo.

```
selected_pcs_bi = [('PC_1', 'PC_2'), ('PC_1', 'PC_3'), ('PC_2', 'PC_3')]
```

```
plt.figure(figsize=(18, 6))
for i, (pc1, pc2) in enumerate(selected_pcs_bi):
    plt.subplot(1, 3, i + 1)
    sns.scatterplot(x=pc1, y=pc2, hue='target', data=pca_train_df, palette='magma')
    plt.title(f'{pc1} vs {pc2} por Clase')
    plt.xlabel(pc1)
    plt.ylabel(pc2)
    # Personalizar las etiquetas de la leyenda
    handles, labels = plt.gca().get_legend_handles_labels()
    plt.gca().legend(handles=handles, labels=[breast_cancer.target_names[0], breast_cancer.target_names[1]])

plt.tight_layout()
plt.show()
```



**Gráficas de Densidad Univariable para Componentes Principales Específicas (PC\_30).**

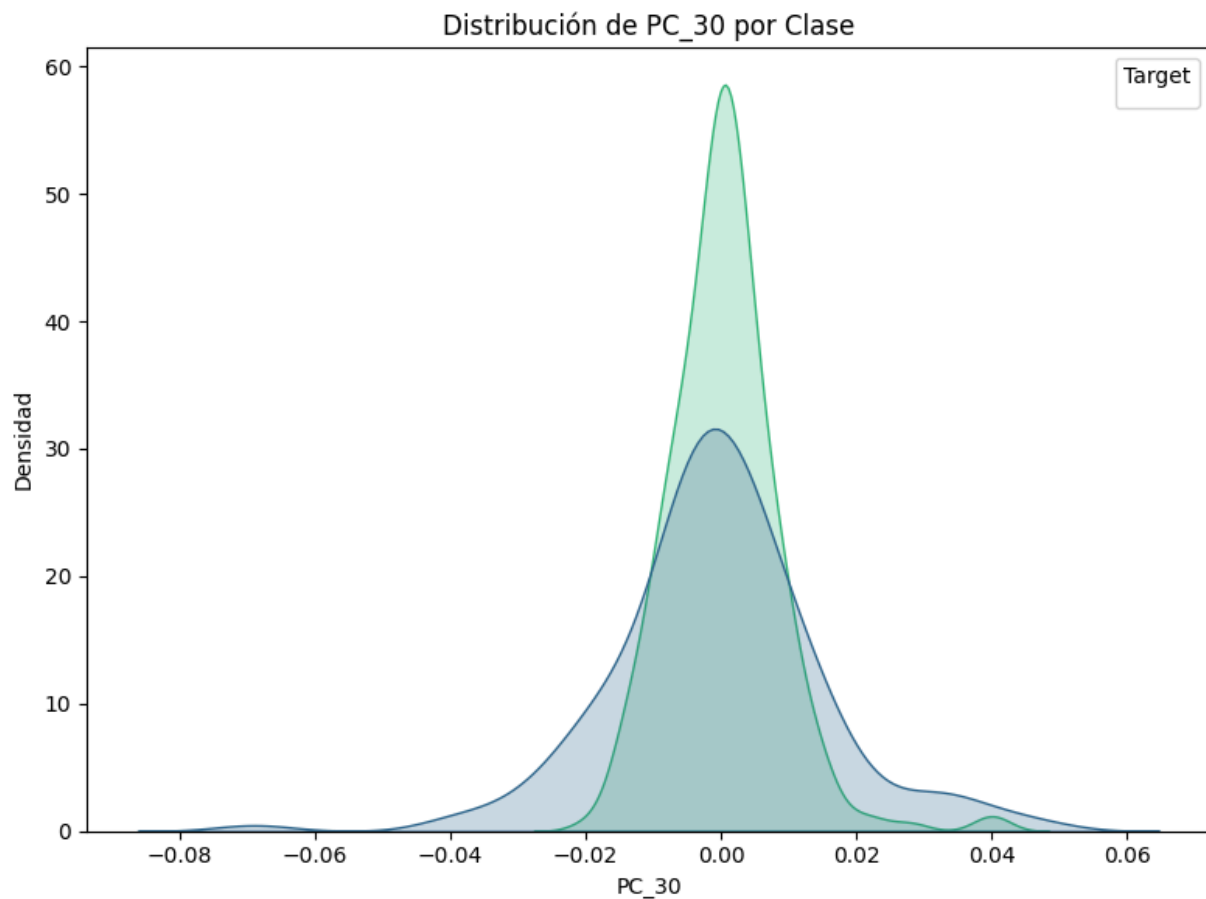
Para asegurar que tenemos todas las componentes (30 en este caso), re-transformamos `X_train_scaled` y usamos el objeto 'pca' que ya fue ajustado con `n_components=None` en el cell id c999288e.

`X_train_pca_full` contendrá todos los componentes principales.

```
X_train_pca_full = pca.transform(X_train_scaled)

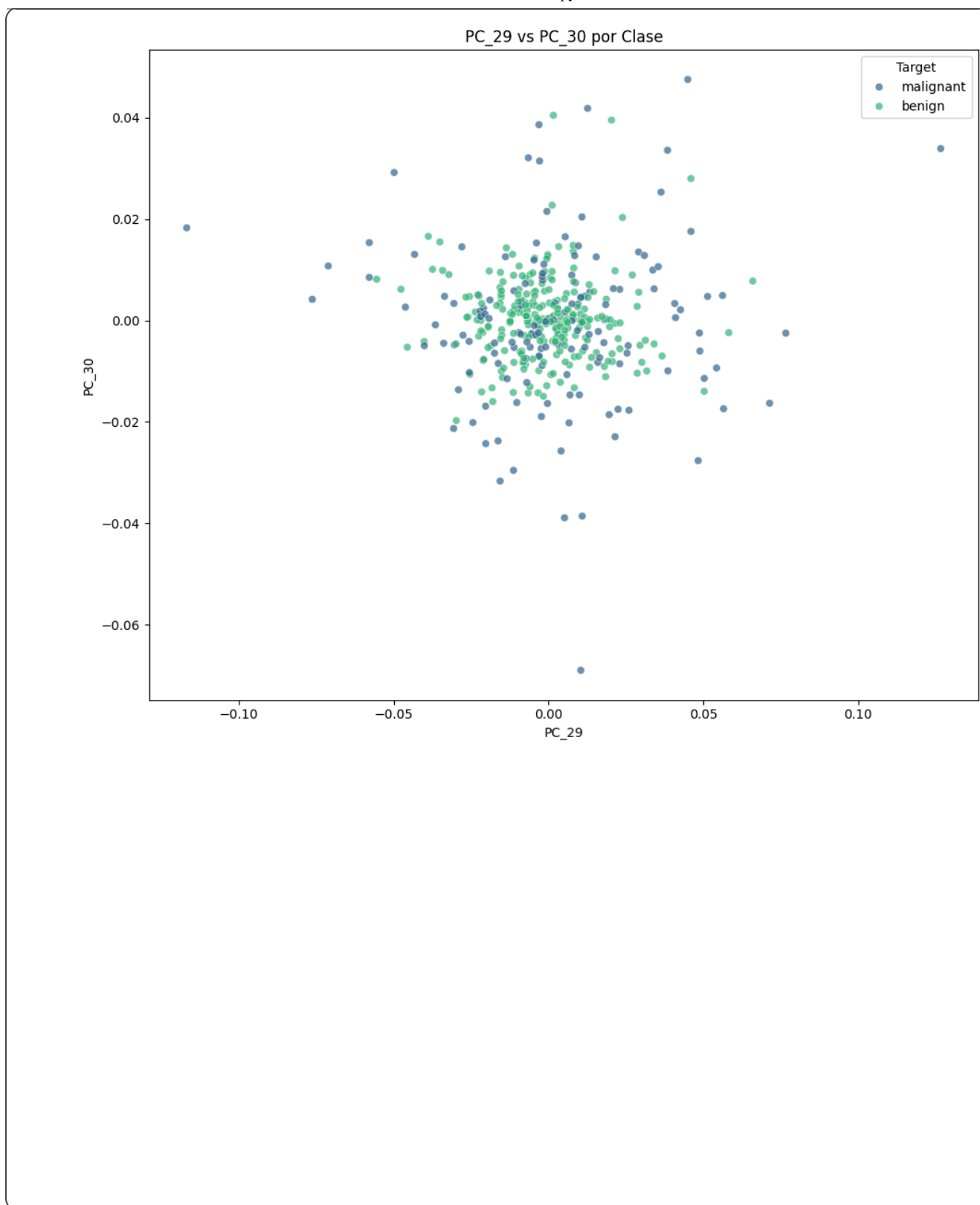
# Crear un DataFrame con todos los componentes principales y la variable
pca_train_df_full = pd.DataFrame(X_train_pca_full, columns=[f'PC_{i+1}'
pca_train_df_full['target'] = y_train.values

# Gráfica de densidad para PC_30
plt.figure(figsize=(8, 6))
sns.kdeplot(data=pca_train_df_full, x='PC_30', hue='target', fill=True,
plt.title('Distribución de PC_30 por Clase')
plt.xlabel('PC_30')
plt.ylabel('Densidad')
current_handles, current_labels = plt.gca().get_legend_handles_labels()
new_labels = [breast_cancer.target_names[int(float(label))]] for label in
plt.gca().legend(current_handles, new_labels, title='Target')
plt.tight_layout()
plt.show()
```



### Gráfica de Dispersión Bivariable para Componentes Principales Específicas (PC\_29 vs PC\_30).

```
# Gráfica de dispersión para PC_29 vs PC_30
plt.figure(figsize=(10, 8))
sns.scatterplot(x='PC_29', y='PC_30', hue='target', data=pca_train_df_fu)
plt.title('PC_29 vs PC_30 por Clase')
plt.xlabel('PC_29')
plt.ylabel('PC_30')
handles, labels = plt.gca().get_legend_handles_labels()
new_labels = [breast_cancer.target_names[int(float(label))]] for label in
plt.gca().legend(handles=handles, labels=new_labels, title='Target')
plt.tight_layout()
plt.show()
```



### Análisis de Cargas (Loadings) de PCA.

Vamos a examinar las cargas (loadings) de los componentes principales. Estas cargas nos indican la correlación entre las características originales y los componentes principales, revelando qué características tienen un mayor peso o contribución en la

formación de cada componente. Esto es crucial para interpretar el significado de cada componente principal.

```
# Crear un DataFrame con las cargas (loadings) de PCA
# Las 'components_' del objeto PCA son las cargas (transpuestas)
loadings = pd.DataFrame(pca.components_,
                        columns=X.columns,
                        index=[f'PC_{i+1}' for i in range(pca.n_components)])

print("Cargas (Loadings) de los Componentes Principales:\n")
display(loadings.head())

# Visualizar las cargas de los primeros componentes principales
num_components_to_plot = 4 # Por ejemplo, las primeras 4 componentes que

plt.figure(figsize=(12, 8))
# Ordenar las características por el valor absoluto de la carga en el primer componente
sorted_loadings = loadings.iloc[:num_components_to_plot].transpose().reindex(sorted_loadings.index, method='nearest')
sns.heatmap(sorted_loadings, cmap='viridis', annot=True, fmt=".2f")
plt.title('Cargas (Loadings) de los Primeros Componentes Principales (Ordenados)')
plt.xlabel('Componentes Principales')
plt.ylabel('Características Originales')
plt.tight_layout()
plt.show()
```





## Cargas (Loadings) de los Componentes Principales:

|      | mean<br>radius | mean<br>texture | mean<br>perimeter | mean<br>area | mean<br>smoothness | mean<br>compactness | coi |
|------|----------------|-----------------|-------------------|--------------|--------------------|---------------------|-----|
| PC_1 | 0.216808       | 0.096076        | 0.225343          | 0.219624     | 0.139359           | 0.239204            | (   |

**Interpretación de las Cargas:**

PC\_3 0.001125 0.172029 -0.001558 0.022899 -0.179138 -0.077662 -(

Observando el heatmap de cargas, podemos identificar qué características originales tienen un mayor impacto en cada componente principal:

PC\_4 -0.046163 0.581055 -0.046175 -0.055141 -0.066108 -0.008045 -(

PC\_5 -0.033594 -0.006157 -0.033229 -0.005554 0.352671 -0.022052 -(

- **PC\_1:** Este componente parece estar fuertemente influenciado por características relacionadas con el tamaño y la severidad del tumor, como mean radius, mean

perimeter, mean area, worst radius, worst perimeter y worst area. Todas estas tienen cargas positivas y altas, lo que indica que PC\_1 es una medida general del tamaño del tumor.

- **PC\_2:** Este componente muestra altas cargas (positivas o negativas) para características como mean texture y worst texture. Esto sugiere que PC\_2 podría estar capturando la variabilidad relacionada con la textura de las células.

- **PC\_3 y PC\_4:** Estos componentes pueden estar capturando aspectos más sutiles de la forma y la irregularidad de las células, como mean fractal dimension, mean concavity o mean symmetry, aunque con cargas generalmente más bajas que los primeros dos componentes.

**Extra: Aplicando PCA a una Imagen.**

Vamos a crear una imagen sintética, aplicarle PCA y luego reconstruirla para ver el efecto de la reducción de dimensionalidad.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA

# 1. Crear una imagen sintética (ej. una imagen en escala de grises 100x
# Podemos crear un patrón simple para que sea más fácil visualizar el ef
img_size = 100
original_image = np.zeros((img_size, img_size))

# Añadir un patrón de gradiente o alguna forma simple
for i in range(img_size):
    original_image[i, :] = np.sin(i / 10.0) * 0.5 + 0.5 # Gradiente hori
```

```
original_image[:, i] += np.cos(i / 8.0) * 0.3 + 0.3 # Gradiente vert

# Normalizar la imagen a valores entre 0 y 1
original_image = (original_image - original_image.min()) / (original_image.max() - original_image.min())

print("Imagen sintética creada.")
```

Imagen sintética creada.

## Preparación de la Imagen para PCA.

Para aplicar PCA, la imagen debe ser "aplanada" en un vector. Si tuviéramos múltiples imágenes, cada una se aplanaría en una fila de un dataset más grande. En este ejemplo, trataremos cada fila de la imagen como una 'muestra' para PCA, lo que nos permitirá ver cómo PCA reduce las características en el eje horizontal.

```
# 2. Aplanar la imagen (en este caso, cada fila es una 'muestra')
# PCA trabajará sobre las columnas (características) de esta matriz
X_image = original_image.copy()

# 3. Aplicar PCA
# Reduciremos la dimensionalidad a, digamos, 10 componentes para este eje
n_components_img = 10
pca_img = PCA(n_components=n_components_img)

# Ajustar PCA y transformar los datos de la imagen
X_transformed = pca_img.fit_transform(X_image)

print(f"Dimensiones de la imagen original: {X_image.shape}")
print(f"Dimensiones de la imagen transformada (con {n_components_img} componentes): {X_transformed.shape}")
```

Dimensiones de la imagen original: (100, 100)  
Dimensiones de la imagen transformada (con 10 componentes): (100, 10)

## Reconstrucción y Visualización.

Ahora, reconstruiremos la imagen a partir de los componentes principales y la compararemos con la imagen original.

```
# 4. Reconstruir la imagen
# Esto se hace invirtiendo la transformación de PCA
reconstructed_image = pca_img.inverse_transform(X_transformed)

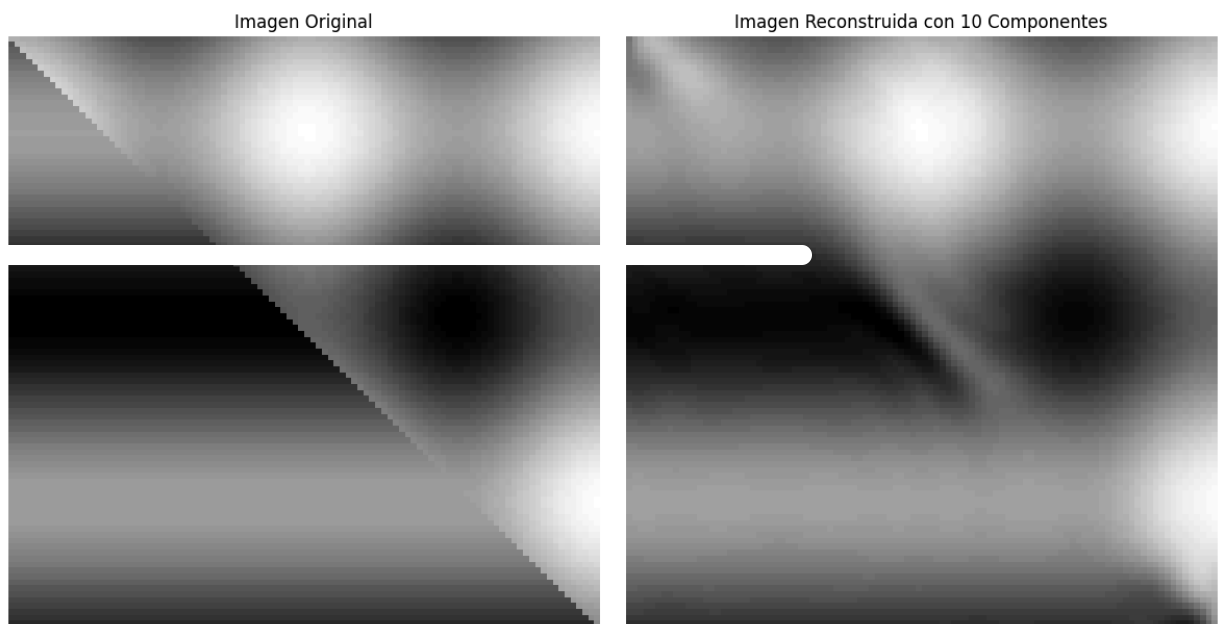
# 5. Visualizar la imagen original y la reconstruida
plt.figure(figsize=(12, 6))
```

```
plt.subplot(1, 2, 1)
plt.imshow(original_image, cmap='gray')
plt.title('Imagen Original')
plt.axis('off')

plt.subplot(1, 2, 2)
plt.imshow(reconstructed_image, cmap='gray')
plt.title(f'Imagen Reconstruida con {n_components_img} Componentes')
plt.axis('off')

plt.tight_layout()
plt.show()

print("Observa cómo la imagen reconstruida captura las características p
```



Observa cómo la imagen reconstruida captura las características principal

# Sistemas de Recomendación.

## Construcción.

```
import pandas as pd

# 1. Definir nombres de columnas para u.data y cargar el dataset
data_cols = ['user_id', 'item_id', 'rating', 'timestamp']
df_data = pd.read_csv('/content/u.data', sep='\t', names=data_cols)

# 2. Definir nombres de columnas para u.item y cargar el dataset
# El archivo u.item contiene metadatos de las películas, incluyendo el t
item_cols = ['movie_id', 'movie_title', 'release_date', 'video_release_d
              'unknown', 'Action', 'Adventure', 'Animation', 'Children\'s
              'Crime', 'Documentary', 'Drama', 'Fantasy', 'Film-Noir', 'H
              'Musical', 'Mystery', 'Romance', 'Sci-Fi', 'Thriller', 'War
df_item = pd.read_csv('/content/u.item', sep='|', names=item_cols, encod

# 3. Fusionar df_data con df_item para asignar los nombres de las pelícu
# La columna 'item_id' en df_data corresponde a 'movie_id' en df_item
df_merged = pd.merge(df_data, df_item, left_on='item_id', right_on='movi

# Eliminar la columna 'movie_id' ya que es redundante después de la fusi
df_merged = df_merged.drop('movie_id', axis=1)

print("Datasets u.data y u.item cargados y fusionados.")
```

Datasets u.data y u.item cargados y fusionados.

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Crear la matriz usuario-item
# Filas: user_id, Columnas: item_id, Valores: rating
# Los valores faltantes se rellenarán automáticamente con NaN por pivot_
user_item_matrix = df_merged.pivot_table(index='user_id', columns='item_

print("Matriz Usuario-Item creada. Primeras 5 filas y columnas:")
display(user_item_matrix.head())

print(f"\nNúmero total de valores NaN en la matriz: {user_item_matrix.is
print(f"Porcentaje de valores NaN en la matriz: {np.mean(user_item_matri
```

Matriz Usuario-Item creada. Primeras 5 filas y columnas:

| item_id | 1   | 2   | 3   | 4   | 5   | 6   | 7   | 8   | 9   | 10  | ... | 1673 | 1674 |
|---------|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|------|------|
| user_id |     |     |     |     |     |     |     |     |     |     |     |      |      |
| 1       | 5.0 | 3.0 | 4.0 | 3.0 | 3.0 | 5.0 | 4.0 | 1.0 | 5.0 | 3.0 | ... | NaN  | NaN  |
| 2       | 4.0 | NaN | NaN | NaN | NaN | NaN | NaN | NaN | NaN | 2.0 | ... | NaN  | NaN  |
| 3       | NaN | NaN | NaN | NaN | NaN | NaN | NaN | NaN | NaN | NaN | ... | NaN  | NaN  |
| 4       | NaN | NaN | NaN | NaN | NaN | NaN | NaN | NaN | NaN | NaN | ... | NaN  | NaN  |
| 5       | 4.0 | 3.0 | NaN | NaN | NaN | NaN | NaN | NaN | NaN | NaN | ... | NaN  | NaN  |

5 rows x 1682 columns

Número total de valores NaN en la matriz: 1486126

Porcentaje de valores NaN en la matriz: 93.70%

```
import random

# 1. Esconder aleatoriamente el 10% de los ratings para evaluación
# Crearemos una copia de la matriz para trabajar con ella
user_item_matrix_eval = user_item_matrix.copy()

# Obtener los índices de los ratings existentes (no NaN)
non_nan_indices = np.argwhere(~np.isnan(user_item_matrix_eval.values))

# Calcular el número de ratings a esconder (10%)
num_ratings_to_hide = int(len(non_nan_indices) * 0.10)

# Seleccionar aleatoriamente los índices a esconder
indices_to_hide = random.sample(list(non_nan_indices), num_ratings_to_hide)

# Esconder los ratings estableciéndolos en NaN
for r, c in indices_to_hide:
    user_item_matrix_eval.iloc[r, c] = np.nan

print(f"Número total de ratings en la matriz original: {user_item_matrix_eval.values.size}")
print(f"Número de ratings escondidos para evaluación: {num_ratings_to_hide}")
print(f"Número de ratings restantes en user_item_matrix_eval: {user_item_matrix_eval.values.size - num_ratings_to_hide}")

print("\nMatriz con ratings escondidos (primeras 5 filas):")
display(user_item_matrix_eval.head())
```

Número total de ratings en la matriz original: 100000  
 Número de ratings escondidos para evaluación: 10000  
 Número de ratings restantes en user\_item\_matrix\_eval: 90000

Matriz con ratings escondidos (primeras 5 filas):

| item_id | 1   | 2   | 3   | 4   | 5   | 6   | 7   | 8   | 9   | 10  | ... | 1673 | 1674 |
|---------|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|------|------|
| user_id |     |     |     |     |     |     |     |     |     |     |     |      |      |
| 1       | 5.0 | 3.0 | 4.0 | 3.0 | 3.0 | 5.0 | 4.0 | 1.0 | 5.0 | 3.0 | ... | NaN  | NaN  |
| 2       | 4.0 | NaN | NaN | NaN | NaN | NaN | NaN | NaN | NaN | NaN | ... | NaN  | NaN  |
| 3       | NaN | NaN | NaN | NaN | NaN | NaN | NaN | NaN | NaN | NaN | ... | NaN  | NaN  |
| 4       | NaN | NaN | NaN | NaN | NaN | NaN | NaN | NaN | NaN | NaN | ... | NaN  | NaN  |
| 5       | 4.0 | 3.0 | NaN | NaN | NaN | NaN | NaN | NaN | NaN | NaN | ... | NaN  | NaN  |

5 rows x 1682 columns

```
# 2. Centrar los ratings por usuario
# Calculamos la media de los ratings para cada usuario (ignorando NaNs)
user_ratings_mean = user_item_matrix_eval.mean(axis=1)

# Restamos la media de cada usuario a sus ratings correspondientes
# Aplicamos esta operación solo a los valores no nulos
user_item_matrix_centered = user_item_matrix_eval.apply(lambda x: x - user_ratings_mean[x], axis=1)

print("\nMatriz con ratings centrados por usuario (primeras 5 filas):")
display(user_item_matrix_centered.head())

print("\nEjemplo: Media de ratings para el usuario 1 antes y después de")
print(f"Media original para usuario 1: {user_item_matrix_eval.loc[1].mean()}")
print(f"Media centrada para usuario 1: {user_item_matrix_centered.loc[1].mean()}")
```

Matriz con ratings centrados por usuario (primeras 5 filas):

| item_id | 1        | 2         | 3        | 4         | 5         | 6        | 7        |
|---------|----------|-----------|----------|-----------|-----------|----------|----------|
| user_id |          |           |          |           |           |          |          |
| 1       | 1.390244 | -0.609756 | 0.390244 | -0.609756 | -0.609756 | 1.390244 | 0.390244 |
| 2       | 0.267857 | NaN       | NaN      | NaN       | NaN       | NaN      | NaN      |
| 3       | NaN      | NaN       | NaN      | NaN       | NaN       | NaN      | NaN      |
| 4       | NaN      | NaN       | NaN      | NaN       | NaN       | NaN      | NaN      |
| 5       | 1.180645 | 0.180645  | NaN      | NaN       | NaN       | NaN      | NaN      |

5 rows x 1682 columns

Ejemplo: Media de ratings para el usuario 1 antes y después de centrar:

Media original para usuario 1: 3.61

Media centrada para usuario 1: -0.00

```
# Instalar fancyimpute si no está instalado (se asume que ya se ejecutó
# !pip install fancyimpute
```

```
from fancyimpute import SoftImpute
from sklearn.metrics import mean_absolute_error, mean_squared_error
```

```
print("Aplicando SoftImpute para la reconstrucción de la matriz...")
```

```
# Inicializar el imputador SoftImpute
# convergence_threshold: si los cambios en la matriz imputada son menor
# max_iters: número máximo de iteraciones.
# SoftImpute está diseñado para completar matrices, así que lo aplicamo
# Aquí usamos user_item_matrix_eval que tiene los NaNs introducidos art
imputer_softimpute_mc = SoftImpute(convergence_threshold=0.001, max_ite
```

```
# Realizar la imputación de la matriz centrada
# SoftImpute trabaja con arrays numpy, por lo que convertimos a .values
user_item_matrix_completed_values = imputer_softimpute_mc.fit_transform
```

```
# Convertir el resultado de nuevo a DataFrame de Pandas con los índices
user_item_matrix_completed = pd.DataFrame(
    user_item_matrix_completed_values,
    index=user_item_matrix_eval.index,
    columns=user_item_matrix_eval.columns
)
```

```
print("Matriz completada con SoftImpute. Primeras 5 filas y columnas re
display(user_item_matrix_completed.head())
```

```
# --- Evaluación de la Reconstrucción en los Ratings Escondidos ---
```

```
# 1. Obtener los ratings originales que fueron escondidos
# Reconstruimos la lista de índices de los ratings que fueron escondidos
# Nota: 'indices_to_hide' fue generado previamente
original_hidden_ratings = []
for r, c in indices_to_hide:
    user_id = user_item_matrix.index[r]
    item_id = user_item_matrix.columns[c]
    original_hidden_ratings.append({'user_id': user_id, 'item_id': item_id})

df_hidden_ratings = pd.DataFrame(original_hidden_ratings)

# 2. Obtener los ratings reconstruidos para estas mismas posiciones
predicted_hidden_ratings = []
for index, row in df_hidden_ratings.iterrows():
    user_id = row['user_id']
    item_id = row['item_id']
    predicted_hidden_ratings.append(user_item_matrix_completed.loc[user_id, item_id])

df_hidden_ratings['predicted_rating'] = predicted_hidden_ratings

print("\nComparación de ratings originales y predichos (primeras 5 filas)")
display(df_hidden_ratings.head())

# 3. Calcular métricas de evaluación
mae = mean_absolute_error(df_hidden_ratings['original_rating'], df_hidden_ratings['predicted_rating'])
rmse = np.sqrt(mean_squared_error(df_hidden_ratings['original_rating'], df_hidden_ratings['predicted_rating']))

print(f"\nMétricas de evaluación para la reconstrucción de ratings escondidos")
print(f"Mean Absolute Error (MAE): {mae:.4f}")
```